Operational semantics for Prolog with Cut in Rocq and its application to determinacy analysis

Davide Fissore
Université Côte d'Azur, Inria, France

Enrico Tassi
Université Côte d'Azur, Inria, France

Abstract

We mechanize two operational semantics for Prolog with cut and prove them equivalent, in the Rocq prover. The first semantics is based on a stack of choice points computations and is well suited for studying optimizations employed by Prolog abstract machines; moreover, it closely models Elpi's implementation. The second semantics is instead based on and-or trees, in which the branches pruned by the cut operator are made explicit.

We use the tree-based semantics (1) to prove properties about the effect of the cut operator in the evaluation of choice points, since the rules from classical logic do not apply, and (2) to reason about the determinacy analysis introduced in Elpi version 3.0, which statically identifies computations that produce at most one result.

2012 ACM Subject Classification Theory of computation → Operational semantics

Keywords and phrases Semantics, Prolog, Elpi, Determinacy Analysis, Rocq, Coq

Digital Object Identifier 10.4230/LIPIcs.CVIT.2016.23

Funding *Davide Fissore*: This work has been supported by the French government, through the France 2030 investment plan managed by the Agence Nationale de la Recherche, as part of the "UCA DS4H" project, reference ANR-17-EURE-0004.

1 Introduction

Elpi is a dialect of λProlog (see [14, 15, 7, 12]) used as an extension language for the Rocq prover (formerly the Coq proof assistant). Elpi has become an important infrastructure component: several projects and libraries depend on it [13, 3, 4, 19, 8, 9]. Examples include the Hierarchy-Builder library-structuring tool [5] and Derive [17, 18, 11], a program-and-proof synthesis framework with industrial applications at SkyLabs AI.

Starting with version 3, Elpi is equipped with a static analysis for determinacy [10] to help users tame backtracking. Rocq users are familiar with functional programming but not necessarily with logic programming, in particular uncontrolled backtracking is a common source of inefficiency and makes debugging harder. The determinacy checkers identifies predicates that behave like functions, i.e., predicates that commit to their first solution and leave no *choice points* (places where backtracking could resume).

This paper reports our first steps towards a mechanization, in the Rocq prover, of the determinacy analysis from [10]. We focus on the control operator *cut*, which is useful to restrict backtracking but makes the semantics depart from a pure logical reading.

We formalize two operational semantics for Prolog with cut. The first is a stack-based semantics that closely models Elpi's implementation and is similar to the semantics mechanized by Pusch in Isabelle/HOL [16] and to the model of Debray and Mishra [6, Sec. 4.3]. This stack-based semantics is a good starting point to study further optimizations used by standard Prolog abstract machines [20, 1], but it makes reasoning about the scope of cut difficult. To address this limitation we introduce a tree-based semantics in which the branches pruned by *cut* are explicit and we prove the two semantics equivalent. Using the

```
Inductive P := IP of nat. Inductive D := ID of nat. Inductive V := IV of nat.

Inductive Tm :=
  | Tm_P of P        | Tm_D    of D
  | Tm_V of V        | Tm_App  of Tm & Tm.
```

Figure 1 Tm types

```
Record Unif := {
  unify : Tm → Tm → Σ → option Σ;
  matching : Tm → Tm → Σ → option Σ;
}.
```

Figure 2 Unif type

tree-based semantics, we then prove some properties about the impact of evaluating the *cut* in query disjunctions. Unlike in classical logic, $q_1 \vee q_2$ is not equivalent to $q_2 \vee q_1$, since q_2 may be cut-away by q_1. Moreover, we show that if every rule of a predicate passes the determinacy analysis, then a call to that predicate does not leave any choice points.

2 Preliminaries: syntax and informal semantics

put unif and program in variables hides from types

Before going to the two semantcis, we show the piece of data structure that are shared by the them. The smallest unit of code that we can use in the langauge is an atom. The atom inductive (see Type 1) is either a cut or a call. A call carries a term (see Figure 1). A term (Tm) is either a predicate, a datum, a variable or the binary application of two terms.

```
Inductive A := cut | call : Tm → A.                          (1)
Record R := mkR { head : Tm; premises : list A }.            (2)
Record ℙ := { rules : seq R; sig : sigT }.                   (3)
Definition Σ := {fmap V → Tm}.                               (4)
Definition bc : ℙ → 𝓕𝓿 → Tm → Σ → 𝓕𝓿 * seq (Σ * seq A) :=    (5)
```

A rule (see Type 2) has a head of type term and a list of premises, that are a list of atoms. A program (see Type 3) is made by its rules and the signatures of predicates, `sigT` is a mapping from predicates to their signatures, signatures represent the defintion of type from the simply typed lambda calculus, i.e. it is either a base type or an arrow for term application. We decorate arrows to know the mode (input or output) of the application argument.

A substitution (see Type 4) is a mapping from variables to terms. It is the output of a successful query. The empty substitution is noted ε.

The backchain function (bc, see Type 5) takes: a program P; a set fv of variables ($\mathcal{F}_v$); a query q; and the substitution σ. A backchain is performed when the current atom is a `call`. In this case, it start by recursively *dereferencing* the variables in σ, i.e. replacing all the variables in the query by their term mapping in σ.

TODO: free variables and program freshness

Then it takes the head of the term, which should be rigid, i.e. a predicate p, and look for the signature s of p in the program. Then, for each rule r in P, it unifies the head of r with q. Unification is possible thanks to the unifcator U, whose type is shown in Figure 2. U explains how to unify terms up to standard unifcation (for output terms) or matching (for input terms). The function *unif* (resp. *matching*) unifies (resp. matches) the two terms under σ, it return an extension of σ if the two terms can be unified (resp. matched), and None otherwise. The result of a backchain is made by filtering the rules in P that can be used on the given query. In particular, for each rule filtered rules r, it returnes a pair containing the extension of σ making the head of r and q unify and the list of premises of r.

```
f 1 2.   f 2 3.    r 0 0.   r 0 1.   r 0 2.
g X Z :- r X Y, f Y Z, !.  % r1
g X Z :- f X Y, f Y Z.     % r2
```

Figure 3 Small ELPI program example

```
Inductive tree :=
  | KO | OK | TA of A
  | Or  of option tree & Σ & tree
  | And of tree & seq A & tree.
```

Figure 4 Tree inductive

2.1 The cut operator

ELPI is a dialect of λPROLOG with the hard cut operator and, in the language we have shown before, *cut* is an atom. The semantics of the cut operator adopted in the ELPI language corresponds to the *hard cut* operator of standard SWI-PROLOG. This operator has two primary purposes. First, it eliminates all alternatives that are created either simultaneously with, or after, the introduction of the cut into the execution state.

se metti r1 = g A B :- f A B. allora g e f sono funzioni, e puoi spiegare anche l'idea del detcheck qui

The ELPI program we will use as an example in the following sections is shown in Figure 3.

To illustrate the high-level description of *cut*, consider the program P shown in Figure 3 and the query $q = \texttt{g 0 R}$. Both rules for `g` can be used on the query q. Backchaining q in P from the empty (ε) subsitution returns the list $[(\sigma_1, [\texttt{r X Y, f Y Z, !}]), (\sigma_2, [\texttt{f X Y, f Y Z}])]$ with σ_1 and σ_1 both equal to $\{\texttt{X} \mapsto \texttt{0}, \texttt{Z} \mapsto \texttt{R}\}$. The two elements of the list are generated respectively from the rules r_1 and r_2.

The next step of the interpretation corresponds to the backchaining of `r X Y` from σ_1. This returns $[(\sigma_3, []), (\sigma_4, []), (\sigma_5, [])]$ with $\sigma_3 := \sigma_1 + \{\texttt{Y} \mapsto \texttt{0}\}$, $\sigma_4 := \sigma_1 + \{\texttt{Y} \mapsto \texttt{1}\}$ and $\sigma_5 := \sigma_1 + \{\texttt{Y} \mapsto \texttt{2}\}$. The three lists of premises are empty since all rules for `r` have no premises.

Backchaining `f Y Z` in σ_3 gives no solution, therefore, by backtracking, we backchain `f Y Z` in σ_4. This gives a solution, namely $\sigma_6 := \sigma_4 + \{\texttt{R} \mapsto \texttt{2}\}$.

We now reach the cut operator and can see its double side behavior. In the current execution state, we have still two unexplored alternatives: we can backtrack on the third solution for `r` that assign `Y` to `2` and on the rules r_2. We can note, however, the first alternative have been generated after the cut introduction, and the second alternative have been generated at the same moment of the cut. Therefore both are cut away leaving no choice point. The final complete solution is: $\{\texttt{X} \mapsto \texttt{0}, \texttt{Z} \mapsto \texttt{R}, \texttt{Y} \mapsto \texttt{1}, \texttt{R} \mapsto \texttt{2}\}$, where `X`, `Y` and `Z` can be ignored since they are the result internal unification.

We propose two formal, operational semantics for a logic program with cut. The two semantics are based on different syntaxes, the first syntax (called tree) exploits a tree-like structure and is ideal both to have a graphical view of its evolution while the tree is being intepreted and to prove lemmas over it. The second syntax, called elpi, is the ELPI's syntax and has the advantage of reducing the computational cost of cutting and backtracking alternatives by using shared pointers. We aim to prove the equivalence of the two semantics together with some interesting lemmas of the cut behavior.

Notations used in the paper In the following section we note $\mathbb{B}$ for the bolean type, and $\top$ and $\bot$ for `true` and `false`. The option instances are noted $\boxed{t}$ for *Some* t and $\square$ for *None*.

3 And-Or Tree semantics

The tree state The tree inductive, shown in Figure 4. We distinguish 5 main cases: *KO*, *OK*, and are special meta-symbols representing, respectively, the basic failed and a successful trees. These symbols are considered meta because they are internal intermediate symbols

```
Fixpoint get_end s A : Σ * tree:=
  match A with
  | TA _ | KO | OK ⇒ (s, A)
  | Or □ s₁ B ⇒ get_end s₁ B
  | Or (Some A) _ _ ⇒ get_end s A
  | And A _ B ⇒
    let (s', pA) := get_end s A in
    if pA == OK then get_end s' B
    else (s', pA)
  end.
```

(a) Defintion of *get_end*

```
Definition get_subst s A := (get_end s A).1.

Definition path_end A := (get_end ε A).2.

Definition success A := path_end A == OK.

Definition failed A := path_end A == KO.

Definition path_atom A :=
    if path_end A is TA _ then ⊤
    else ⊥.
```

(b) Functions from *get_end*

Figure 5 *get_end* and derives functions

used to give structure to the tree.

The *TA* constructor (acronym for tree-atom) is the constructor of atoms in the tree. They are either the cut operator or a call, in these cases, the interpreter should evaluate the atom by either cutting the subtrees in the scope of the cut or backchaining the call in the program.

The two recursive cases of a tree are the *Or* and *And* nodes. The *Or* node $A \vee B_\sigma$ denotes a disjunction between two trees A and B. The first branch is optional, if absent it represents a dead tree, i.e. a tree that has been entirely explored. The second branch is annotated with a suspended substitution σ so that, upon backtracking to B, σ is used as the initial substitution for the execution of B.

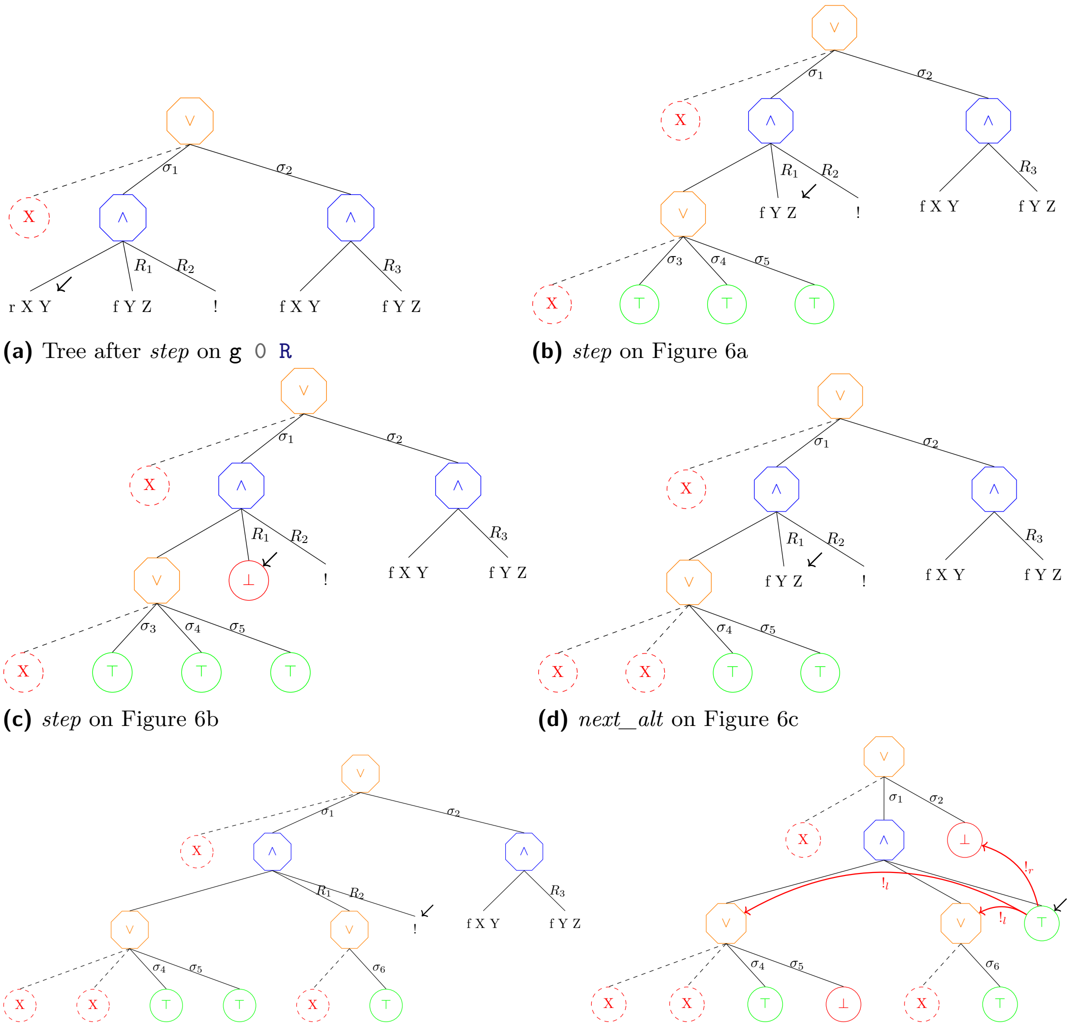
The *And* node $A \wedge_{B_0} B$ represents a conjunction of two trees A and B. We call B_0 the reset point for B; it is used to restore the state of B to its initial form if a backtracking operation occurs on A. An example of this behavior is shown in Section 3.2

A graphical representation of a tree is shown in Figure 6a. To make the graph more compact, the *And* and *Or* nodes are n-ary rather than binary, and the children of these nodes associate to the right. The *KO* node acts as the neutral elements in the *Or* list, while *OK* is the neutral element of the *And* list.

The tree interpreter The interpretation of a tree is performed by two main routines: *step* and *next_alt* that traverse the tree depth-first, left-to-right. Then, then *runT* inductive makes the transitive closure of step *step* and *next_alt*: it iterates the calls to its these auxiliary functions as much as needed. In Types 7–9 we give the types contrats of these symbols where $\mathcal{F}_v$ is a set of variable names.

```
Inductive tag := Expanded | CutBrothers | Failed | Success.          (6)
Definition step : ℙ → 𝓕ᵥ → Σ → tree → (𝓕ᵥ * tag * tree) :=          (7)
Definition next_alt : 𝔹 → tree → option tree :=                       (8)
Inductive runT (p : ℙ): 𝓕ᵥ → Σ → tree
          → option (Σ * option tree) → 𝔹 → 𝓕ᵥ → Prop :=                (9)
```

The tree interpreter, as in prolog, explores the tree in DFS strategy, to discover the substitution and the leaf of the tree that should be interpreted. These two pieces of information can be retrieved for a substitution s and a tree A thanks to the *get_end* routine, shown in Figure 5a If the tree is already a leaf, *get_end* returns its inputs. Otherwise, if the tree is a disjunction, the path continues on the left subtree, if it exists, otherwise *get_end* contues on the ths using the substitution stored in the *Or* node. In the case of a conjunction, if *get_end* of the lhs returns the tree *OK*, then the *get_end* continues in the rhs, otherwise



(a) Tree after *step* on `g 0 R` **(b)** *step* on Figure 6a

(c) *step* on Figure 6b **(d)** *next_alt* on Figure 6c

(e) *step* on Figure 6d **(f)** *step* on Figure 6e

Figure 6 Some tree representations[1]

we return the result of the lhs. We derive some useful definition from *get_end*. They are listed in Figure 5b.

We have pointed by a black arraw the *path_end* of all the trees in Figure 6.

3.1 The *step* procedure

The *step* procedure takes as input a program, a set of variables, a substitution, and a tree, and returns an updated set of variables, a *tag*, and an updated tree.

The set of variables v given in input to step are used and updated when a backchaining operation takes place. A call to *step* produces a good result if v contains all the variables in the tree. This guarantees that any call to *bc* returns a list of atoms whose variables are disjoint from the variables of the current working tree.

not true in figure:fix + also fix in Section 2.1

[1] The substitution $\sigma_1 \ldots \sigma_6$ are from Section 2.1, R_1, R_2 and R_3 are the reset points and correspond

The *tag* (see Type 6) indicates the kind of an internal tree step: `CutBrothers` denotes the interpretation of a superficial cut, i.e., a cut whose parent nodes are all *And*-nodes. `Expanded` denotes the interpretation of non-superficial cuts or predicate calls. `Failure` and `Success` are returned for, respectively, *failed* and *success* trees.

The step procedure is intended to evaluate atoms, that is, it leaves unchanged the tree iff its *path_end* is not an atom, i.e. if it is a success- or a failed-tree.

```
Lemma succ_step_iff: ∀ p v s A, success A ↔ step p v s A = (v, Success, A).
```
(1)

```
Lemma fail_step_iff: ∀ p v s A, failed A ↔ step p v s A = (v, Failed, A).
```
(2)

The interpretation of a call to a term c stars by calling the bc function on c. The result of the backchain is a list l. If l is empty then the current call is replaced by the *KO* node, otherwise it is replaced by a right skewed tree made by *Or* inner-nodes and each leave correspod to a list of atom e in l. If e is empty then it is mapped to the *OK* tree, otherwise it is mapped to a right-skewed tree made by *And* inner-nodes.

sottolineare sostituzioni

In Figure 6a, we provide a clear example of a call to *step* on the query `q 0 R` on the program P in Figure 3. Let c_0, c_1 and c_2 be the children of the root. c_0, by construction is the *None* tree, while c_1 and c_2 correspod respectively to the premises of rules r_1 and r_2 in P. We need the first *None* subtree so that we can attach a substituion to c_1. In particular, we note that the c_1 (resp. c_2) are related to the substitution σ_1 (resp. σ_2). The value of σ_1 and σ_2 are the same as in Section 2.1. The reset points in the tree are marked with the R symbol. $R_1 \coloneqq$ [`f Y Z, !`], $R_2 \coloneqq$ [`!`] and $R_3 \coloneqq$ `f Y Z`, they are essentially the list of atoms that allowed to generate the current subtree. Other example of call to the *step* procedure triggering a backchain are Figures 6b, 6c, and 6e.

The interpretation of a cut has three main impacts: at first the cut is replaced by the *OK* node, then some special subtrees, in the scope of the *Cut*, are cut away: in particular we need to soft-kill the left-siblings of the *Cut* and hard-kill the right-uncles of the the *Cut*.

▶ **Definition 1** (Left-siblings (resp. right-sibling) and Right-uncles)**.** *Given a node A, the left-siblings (resp. right-sibling) of A are the list of subtrees sharing the same parent of A and that appear on its left (resp. right). The right-uncles of A are the list of right-sibling of the father of A.*

▶ **Definition 2** (Hard-kill, $!_r$ and Soft-kill, $!_l$)**.** *Given a tree t, hard-kill replaces the given subtree with the* KO *node. If t is a success-tree, then soft-kill replaces with* KO *all subtrees that are not part of the path in t leading to the* OK *node.*

An example of the impact of the cut is show in Figure 6f. In Figure 6e, we see that the *path_end* is a cut. The cut is replaced by the *OK* node and it need to cut-away some alternatives in the tree. The red arrows indicate the scope of the cut, they are labeled with $!_r$ and $!_l$ to explain which kind of cut has been performed on each pointed subtree. We note that, after the evaluation of a cut, the path leading to the cut node is not affected by $!_l$, the success under the substitution σ_4 is kept, while the success under the substitution σ_5 is replaced by the *KO* node. If we refer to the paragraph in Section 2.1, we are discarding the third rule of the `r` predicate.

respectively to the list of atoms [`f Y Z, !`], [`!`] and [`f Y Z`]

3.2 The *next_alt* procedure

The *step* procedure alone is not sufficient to reproduce entirely the behavior of the full expected prolog interpreter. As proved in Lemma 1 and Lemma 2, *step* does not make any progression on the tree being interpreted. To solve this problem, we need a routine capable of backtracking in case of failures. This routine return the backtracked tree if it exists, *None* otherwise.

In Figure 6c, we are facing a failure: the interpreter has evaluated the atom `f X Y` under the substitution σ_3 which assigns `Y` to `0`, but, in Figure 3, we have no rule satisfying this query. The role of *next_alt*, in this example, is to kill the path leading to this failure, while reseting the failure to its original shape. We see how the reset point plays a major role in this case. At first we are killing the *OK* node under the subsitution σ_3, since this tree have been entirely explored and led to no solution, then we replace the *KO* node with the information stored in R_1. That is, thanks to R_1, we are restoring the `f Y Z` call, so that we are able to evaluate this node under the subsitutution σ_4. More concretely, we have uncessefully tried rule `r 0 0`, therefore, we need to continue by trying rule `r 0 1`.

Furthermore, *next_alt* accomplishes to another important task. Its behavior changes depending to the value of the bolean it receives in entry: next_alt *false A* aims to recursively cancel all the failures (if any) in *A*, whereas, next_alt *true A* aims to cancel the first success in *A*. This second behavior is helpful since, we need to progress the tree in case of success. For example, consider the tree in Figure 6f, it contains a success, mapping namely `R` to `2`. The interpreter should return this sucess, but, due to the "non-deterministic" behavior of the prolog system, we also need to return all the other solution that could be available by evaluating the rest of the tree. After the replacement of the *path_end* in Figure 6f with *KO*, we remain with a tree having all failing path. The *next_alt* procedure keeps removing them until discovering that the tree have no worth-to explore paths, therefore it will return *None*.

Some interesting property of *next_alt* are shown below and allow to see how *next_alt* complements *step* wrt failed, success and *path_atom* trees.

```
Lemma path_atom_next_alt_id b A: path_atom A → next_alt b A = Some A.          (3)
Lemma next_alt_failedF b A A': next_alt b A = Some A' → failed A' = ⊥.         (4)
Lemma next_altFN_fail: ∀ A, next_alt ⊥ A = ☐ → failed A.                       (5)
```

3.3 The *runT* inductive

The inductive *runT* is modeled as a function taking 4 inputs and returning 3 outputs. The inputs are the program, a set of variables v_0, an initial substitution σ_0, and a tree t_0. The outputs are an optional object r containing a substitution σ_1 and an optional tree t_1, and a boolean b and a set of updated variables v_1. The very interesting piece of information is the result r, if *None*, it tells that the interpretation failed, otherwise, σ_1 represents the most-general unificator that makes the execution of the tree t_0 under the initial substitution σ_0 succeeds: σ_1 is an extension of σ_0. The tree t_1 is the updated tree containing the alternatives that have not yet been explored, if these alternatives do not exists, t_1 is *None*. The output b and v_1 are there to tell respectively if during the execution a superficial cut has been evaluated, and what is the updated set of variables after the interpretation.

replace None with ⊥

The procedure *runT* is based on four main derivation rules, shown in Figure 7. (StopT) If the *path_end* of the tree t_0 is a success, the substitution σ_1 is retrived in t_0 from σ_0 and the input tree is cleaned of its successful path. (StepT) If the *path_end* of t_0 is an atom, then *step* is invoked to evaluate t_0, and *runT* is recursively called on the new tree. (BackT)

If the *path_end* of t_0 is a failure, *next_alt* is called to clear the failed path; if the resulting cleaned tree t_1 exists, *runT* is recursively called on it. Finally, (FAILT) takes into account tree with no solutions.

```
Lemma runT_det: ∀ p v0 s0 t0 r1 r2 b1 b2 v1 v2,
  runT p v0 s0 t0 r1 b1 v1 → runT p v0 s0 t0 r2 b2 v2 → r2 = r1 ∧ v2 = v1 ∧ b2 = b1.    (6)
```

The lemma above tells that *runT* behaves deterministically. The proof is by induction on the first *runT* execution and the proof is rather immediate, since all rules are in exclusive due to the hypothis on the *path_end*: the hypotheses *success* t_0, path_atom t_0 and *failed* t_0 are exclusive. Moreover, by Lemma 6, we have that the hypothesis of FAILT implies *failed* t_0, therefore it is exclusive wrt STOPT and STEPT, but also from BACKT since the have different expectations on the output of *next_alt*.

$$\frac{\text{success } t_0 \qquad \text{get\_subst } s_0\ t_0 = s_1 \qquad \text{next\_alt } \top\ t_0 = t_1}{\text{runT } v_0\ s_0\ t_0\ (\text{Some } (s_1, t_1))\ \bot\ v_0}\ \textsc{StopT}$$

$$\frac{\begin{array}{cc}\text{path\_atom } t_0 & b_2 = \text{is\_cb tg} \mid\mid b_1 \\ \text{step p } v_0\ s_0\ t_0 = (v_1, \text{tg}, t_1) & \text{runT } v_1\ s_0\ t_1\ \text{r}\ b_1\ v_2\end{array}}{\text{runT } v_0\ s_0\ t_0\ \text{r}\ b_2\ v_2}\ \textsc{StepT}$$

$$\frac{\text{failed } t_0 \qquad \text{next\_alt } \bot\ t_0 = \text{Some } t_1 \qquad \text{runT } v_0\ s_0\ t_1\ \text{r n } v_1}{\text{runT } v_0\ s_0\ t_0\ \text{r n } v_1}\ \textsc{BackT}$$

$$\frac{\text{next\_alt } \bot\ t_0 = \square}{\text{runT } v_0\ s_0\ t_0\ \square\ \bot\ v_0}\ \textsc{FailT}$$

Figure 7 Rule system for *runT*

3.4 Properties of *runT*

Working with *runT* allows to prove some interesting properties about the behavior of the cut operator wrt disjunction, i.e. the *Or* node. We have proven several properties that are avalable at *this* link. Below we show two of these lemmas.

▶ **Lemma 3** (runSST_or). $\forall p\ v_0\ v_1\ s_0\ s_1\ t_0\ t_0'\ s_m\ t_1,$

$$\texttt{runT}\ p\ v_0\ s_0\ t_0\ \boxed{s_1, \boxed{t_0'}}\ \top\ v_1 \to \texttt{runT}\ p\ v_0\ s_0\ (\mathit{Or}\ \boxed{t_0}\ s_m\ t_1)\ \boxed{s_1, \boxed{\mathit{Or}\ \boxed{t_0'}\ s_m\ \mathit{KO}}}\ \bot\ v_1$$

```
Lemma runSST_or: ∀ p v0 v1 s0 s1 t0 t0' sm t1,
  runT p v0 s0 t0 (Some (s1, Some t0')) ⊤ v1 →
    let sR := Some (Or (Some t0') sm KO) in
    runT p v0 s0 (Or (Some t0) sm t1) (Some (s1, sR)) ⊥ v1.    (7)
```

```
Lemma run_orSST p v0 v2 s0 s1 sm t0 t0' t1 t1':
  runT p v0 s0 (Or (Some t0) sm t1) (Some (s1, (Some (Or (Some t0') sm t1')))) ⊥ v2 →    (8)
  (∃ b, runT p v0 s0 t0 (Some (s1, Some t0')) b v2 ∧ t1' = if b then KO else t1).
```

In the first lemma we claim that if we run a tree t_0, and the interpretation returns a success after executing a superficial cut (note the `true` boolean), the interpretation of the tree $t_0 t_1$ still return a success, with the same σ_1, the output tree still exists, but the rhs has been cut-away due to the execution of the superficial cut in t_0.

```
Definition stepE v₀ t s a gl ≔
  let (v₁, rs) ≔ bc p v₀ t s in
  (v₁, save_alts a gl (r₂a rs)).
```

```
Inductive runE : ℱᵥ → alts → option (Σ * alts) → Prop ≔        (10)
```

$$\frac{}{\text{runE } v_0\ ((s_0, [::])\ ::\ \text{a})\ (\text{Some } (s_0,\ \text{a}))}\ \textsc{StopE}$$

$$\frac{\text{runE } v_0\ ((s_0,\ \text{gl})\ ::\ \text{ca})\ \text{r}}{\text{runE } v_0\ ((s_0,\ [::\ (\text{cut},\ \text{ca})\ \&\ \text{gl}])\ ::\ \text{a})\ \text{r}}\ \textsc{CutE}$$

$$\frac{\text{stepE } v_0\ \text{t}\ s_0\ \text{al gl} = (v_1,\ [::\ \text{b}\ \&\ \text{bs}\ ]) \qquad \text{runE } v_1\ (\text{b}\ ::\ \text{bs++al})\ \text{r}}{\text{runE } v_0\ ((s_0,\ [::\ (\text{call t},\ \text{ca})\ \&\ \text{gl}])\ ::\ \text{al})\ \text{r}}\ \textsc{CallE}$$

$$\frac{\text{stepE } v_0\ \text{t}\ s_0\ \text{al gl} = (v_1,\ [::]) \qquad \text{runE } v_1\ \text{al r}}{\text{runE } v_0\ ((s_0,\ [::\ (\text{call t},\ \text{ca})\ \&\ \text{gl}])\ ::\ \text{al})\ \text{r}}\ \textsc{BackE}$$

$$\frac{}{\text{runE } v_0\ [::]\ \square}\ \textsc{FailE}$$

Figure 8 Rule system for *runE*

Conversely, if the term $t_0 \vee t_1$ evaluates to a subsitution σ_0 and know that t_0 leaves alternatives (namely t_0'), then we know that evaluating t_0 alone will produce the same substitution σ_0 and alternatives t_0', but, depending on if the evaluation of t_0 has crossed a superficial cut, we know if the state t_1 will be cut-away or not in the evaluation of $t_0 \vee t_1$.

dire che se non ci sono cut superficiali allora si comporta come or della logica normale.

4 Stack semantics

We now introduce the Elpi semantics. The interpreter is close to that of the Elpi language and provides an operational semantics that is similar to the one proposed by Pusch in [16], which is in turn closely related to the semantics given by Debray and Mishra in [6, Section 4.3]. Pusch mechanized this semantics in Isabelle/HOL, together with several optimizations that are also present in the Warren Abstract Machine [20, 1].

We represent a state of the Elpi language thanks to the two mutual inductives shown below.

```
Inductive alts ≔ no_alt | more_alt of (Σ * goals) & alts
with goals ≔ no_goals | more_goals of (A * alts) & goals .
```

An Elpi state is an enhanced two-dimensional list. The outermost list represents a list of alternatives in disjunction, each accompanied by the substitution that should be used for their interpretation. The innermost list is a list of atoms, that is, the list of goals in conjunction. These goals are decorated with a pointer to an alternative list, which is used to keep track of where to jump when a cut is interpreted. We call these special alternatives the *cut-to* alternatives.

In our implementation of

The idea of the Elpi interpreter is to receive a list of alternatives. The first alternative consists of a list of goals. Four cases must be taken into account; they are shown in

Figure 8. In order to simplify goal retrieval, we split the head of the alternatives from the tail, so that it can be immediately matched in the inductive definition. Note that an empty list of alternatives represents, by definition, a failing elpi state. If the goal list is empty (StopE), then we have, by definition, a success, and the input solution together with the list of alternatives is returned. If the goal list starts with a cut (CutE), then the current alternatives are erased in favour of the cut alternatives, and a recursive call is made on the remaining goal list.

Finally, we must consider the case in which the goal list starts with a call. The call can either fail (FailE) or succeed (CallE). We distinguish the two cases by looking if the backchaining operation returns zero or more rules. We have wrapped this task in the *stepE* procedure, which also updates the goal and cut-alternative list. The fail case, is relatively easy: the first goal does not succeed, we need to take the head of the alternatives, and make it the new list of goals to be explored.

The case in which backchaining produces a non empty list, the *save_alts* routine is in cahrge of: taking the list of premises and add to each atom the the list of alternatives a as their new cut-alternatives, then it append the list of goals gl to each of these new lists.

4.1 XXX

It is not possible to prove the same lemma as in section xxx, because no structure, the scope of the cut ... spiegare bene il problema so that i can reuse this subsection has a hook when I will say that l2t is difficult + Yves

5 Equivalence of the two semantics

The equivalence between the two semantics is possible under two conditions: we need to work with "valid tree", i.e. all of those trees that can be generated from a call. Secondly, we need to translate trees into elpi states. In the next to subsection we propose the two functions *valid_state* and *t2l*.

5.1 Valid trees

The inductive tree allows one to generate a large number of trees, some of which are not valid, in the sense that they cannot be produced starting from a given query. The class of valid trees is characterized by the function shown in Figure 9a.

While all base cases of tree are considered valid, we need to analyze carefully the cases for the *Or* and *And* constructors.

For the *Or* constructor, we distinguish two cases depending on whether the left-hand side (lhs) exists. If it does not exist, then the right-hand side (rhs) must be a valid tree. Otherwise, the lhs must itself be a valid tree, and the rhs is either the *KO* tree, since it may have been removed by the evaluation of a superficial cut in the lhs, or it has not yet been explored. In the latter case, it is a `base_or` tree, namely the right-skewed tree formed by a disjunction of conjunctions that is genereated after a succeful backchain to a call.

For the *And* constructor, the lhs is required to be a valid tree. The shape of the rhs depends on whether the lhs is a success. If the lhs is not successful, then the rhs has never been explored: the procedures *step* and *next_alt* modify the rhs only when the lhs succeeds. In this case, the lhs must be the right-skewed tree containing conjunctions of premises atoms, the reset point B_0 ensure what shape the rhs should have. If the lhs is a success tree, then the rhs can be modified by *step* and *next_alt*, therefore it must be a valid tree.

```
Fixpoint valid_tree A :=
  match A with
  | TA _ | OK | KO ⇒ ⊤
  | Or □ _ B ⇒ valid_tree B
  | Or (Some A) _ B ⇒ valid_tree A &&
        ((B == KO) || B.base_or B)
  | And A B0 B ⇒ valid_tree A &&
      if success A then valid_tree B
      else B == big_and B0
  end.
```

(a) Valid tree

```
Fixpoint t2l A s0 bt :=
  match A with
  | OK            ⇒ [:: (s0, [::])]
  | KO            ⇒ [::]
  | TA a          ⇒ [:: (s0, [:: (a, [::]) ])]
  | Or □ s1 B ⇒ add_ca_deep bt (t2l B s1 [::])
  | Or (Some A) s1 B    ⇒
      let lB := t2l B s1 [::] in
      let lA := t2l A s0 lB in
      add_ca_deep bt (lA ++ lB)
  | And A B0 B    ⇒
      let lA  := t2l A s0 bt in
      let lB0 := a2g B0 in
      let lA  := add_deep bt lB0 lA in
      if lA is [:: (s0, gs) & al] then
        let al := map (catr lB0) al in
        let lB := t2l B s0 (al ++ bt) in
        map (catl gs) lB ++ al
      else [::]
end.
```

(b) Tree to list

Figure 9 Valid tree and Tree to list

5.2 From trees to lists

The translation of a tree to a list is shown in Figure 9b. It takes the tree to be translated, a substitution (called s), a list of alternatives, called *bt*. The substitution s tells what is the subsitution of the alternative we are building and is updated when going to the rhs of the *Or* constructor. The *bt* list represents the future alternatives list that have the same depth of the current tree root. They are useful to know if they should be added to the current goal as cut alternatives.

An *OK* node represent a success in a tree, then the translation of this tree returns a singleton list with the cuple $(s, [::])$: there is one alternative with 0 goals, i.e. a success in elpi by StopE. The *KO* node represent a failure, and, therefore 0 alternatives. An atom is translated into a singleton with the atom as first goal and the empty cut-alternative list: we are not adding *bt* since they are the alternatives for the right-sibling, not the right-uncles, this means that if we have a is a cut, then it erases the alternatives bt.

In a disjunction, we are scendendo di un livello the distance from our backtracking list. This means that *bt* becomes the list of right-uncles of the two branches of the *Or* constructor. The compilation of the rhs is done independently of if the lhs exists: we transform the rhs into an elpi state and passing the empty list of alternatives, since the rhs as no right-siblings. Then, thanks to `add_ca_deep`, we recursively add *bt* to every cut-alternative appearing in the translated goals. If the lhs exists, we translate it and pass the translation of the rhs as the list of right-siblings. Then we prepend the translated lhs to the translated rhs and add *bt* with `add_ca_deep`.

We compile the *And* case by translating its lhs to the list lA and reset point to the list $lB0$. Then, thanks to `add_deep`, we recrusively append $lB0$ to the alternatives born in lA, i.e. we leave unchanged the pointers to bt, if any, in lA. We finally need to treat cases whenever the list lA is empty or not. In the former case the empty list is returned, otherwise,

$$
\begin{tikzcd}
t_1 \arrow[r, "*"] \arrow[d, "\mathrm{t2l}"'] & t_2 \arrow[d, "\mathrm{t2l}"] \\
e_1 \arrow[r, "*"] \arrow[ur, "\text{induction}"] & e_2
\end{tikzcd}
$$

Figure 10 Induction scheme for Lemma 10

we hasve a list $[:: (s_0, gs)\&al]$. By the semantics of the *And* in the *runT* interpreter, the rhs of the *And* represent the sequent of goals to be executed after the first alternative in the lhs, it corresponds to the list of goals gs. The reset point $B0$ is to be appended to the next alternatives in the lhs, that is the queue of lA, that is al.

alternative tree? rephrase

We note therefore that the compilation of lB is done by passing the alternatives $al++bt$ since the alternatives in al are to be...

5.3 Equivalence theorems

We have now all the ingredients that ...

```
Lemma tree_to_elpi fv A s₁ b fv' r:
  vars_tree A `<=` fv → vars_sigma s₁ `<=` fv →
  valid_tree A →
    runT u p fv s₁ A r b fv' →
      let xs := t₂l A s₁ [::] in
      let r' := omap (fun '(s, t) ⇒ (s, (t₂l (odflt KO t) s₁ [::]))) r in
      runE u p fv xs r'.
```
(9)

```
Lemma elpi_to_tree v₀ a r :
  runE u p v₀ a r →
  ∀ s₀ t, valid_tree t → t₂l t s₀ [::] = a →
  ∃ b v₁,
  if r is Some (s₁, a') then
    ∃ t₁, runT u p v₀ s₀ t (Some (s₁, t₁)) b v₁ ∧ t₂l (odflt KO t₁) s₀ [::] = a'
  else runT u p v₀ s₀ t □ b v₁.
```
(10)

The proof of Lemma 10 is based on the idea explained in [2, Section 3.3]. An ideal statement for this lemma would be: given a function `l2t` transforming an elpi state to a tree, we would have have that the the execution of an elpi state e is the same as executing *runT* on the tree resulting from `l2t`(e). However, it is difficult to retrive the strucutre of an elpi state and create a tree from it. This is because, in an elpi state, we have no clear information about the scope of an atom inside the list and, therefore, no evident clue about where this atom should be place in the tree.

Our theorem states that, starting from a valid tree t which translates to a list of alternatives $(\sigma_1, g) :: a$. If we run in elpi the list of alternatives, then the execution of the tree t returns the same result as the execution in elpi. The proof is performed by induction on the derivations of the elpi execution. We have 4 derivations.

We have 4 case to analyse:

6 Case study: determinacy alanysis

we mechanize the first order part of xxx.

snippet det, main thm, invariant det tree (valid tree prev section?)

proof induciton on exec, step/next alt preserving invariant proved by induction on the tree.

with list semantics cut and next alt requires to express a link btween the ca or next alts and the current goal, which is non trivial without an intermediate data strature like the tree

7 Related work

The only mechanization of Prolog's semantics we could find is the one by Pusch in Isabelle/HOL [16] that is based on the model of Debray and Mishra [6, Sec. 4.3]. Their work start from that semantics and refines it by introducing some optimizations usually found in the Warren abstract machine [20, 1]. They do not use the semantics to prove properties on the execution of programs as we do in our use case, hence they do not face the difficulty described in seciton XXX that pushed us to develop an more explicit semantics (with respect to cut). In some way our work is complementary, showing he stack based semantics equivalent to more optmized one, we could consider implement in Elpi.

Another relevant work are the ones by King cite, but we could not find their code. Their semantics is denotational and cannot easily cover all the features our paper XX does, but would have been sufficient for this first step.

The proof technique we use to relate the two semantics is inspired by XXX and we thank Y.Bertot for insightful discussions on the topic. In XXX Bertot relates bla bla, witout never constructing a decompiler, exactly as we never build a list to tree function.

8 Conclusion

We do this. This is a first step for XXXX. We need to add substitutions. The missing part of the proof becomes related to abstract interpretation, since the det types for a lattice and one has to prove that the concrete runtime values variables take agree with the abstraction computed statically. This proof is ongoing and somehow orthogonal to the current one since the cut operator and the HO feature do not interfere with eachother in complex ways.

References

1 Hassan Aït-Kaci. *Warren's Abstract Machine: A Tutorial Reconstruction*. The MIT Press, 08 1991. `doi:10.7551/mitpress/7160.001.0001`.

2 Yves Bertot. A certified compiler for an imperative language. Technical Report RR-3488, INRIA, September 1998. URL: `https://inria.hal.science/inria-00073199v1`.

3 Valentin Blot, Denis Cousineau, Enzo Crance, Louise Dubois de Prisque, Chantal Keller, Assia Mahboubi, and Pierre Vial. Compositional pre-processing for automated reasoning in dependent type theory. In Robbert Krebbers, Dmitriy Traytel, Brigitte Pientka, and Steve Zdancewic, editors, *Proceedings of the 12th ACM SIGPLAN International Conference on Certified Programs and Proofs, CPP 2023, Boston, MA, USA, January 16-17, 2023*, pages 63–77. ACM, 2023. `doi:10.1145/3573105.3575676`.

4 Cyril Cohen, Enzo Crance, and Assia Mahboubi. Trocq: Proof transfer for free, with or without univalence. In Stephanie Weirich, editor, *Programming Languages and Systems*, pages 239–268, Cham, 2024. Springer Nature Switzerland.

5 Cyril Cohen, Kazuhiko Sakaguchi, and Enrico Tassi. Hierarchy Builder: Algebraic hierarchies Made Easy in Coq with Elpi. In *Proceedings of FSCD*, volume 167 of *LIPIcs*, pages 34:1–34:21, 2020. URL: `https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.FSCD.2020.34`, `doi:10.4230/LIPIcs.FSCD.2020.34`.

6 Saumya K. Debray and Prateek Mishra. Denotational and operational semantics for prolog. *J. Log. Program.*, 5(1):61–91, March 1988. doi:10.1016/0743-1066(88)90007-6.

7 Cvetan Dunchev, Ferruccio Guidi, Claudio Sacerdoti Coen, and Enrico Tassi. ELPI: fast, embeddable, λProlog interpreter. In *Proceedings of LPAR*, volume 9450 of *LNCS*, pages 460–468. Springer, 2015. URL: https://inria.hal.science/hal-01176856v1, doi:10.1007/978-3-662-48899-7_32.

8 Davide Fissore and Enrico Tassi. A new Type-Class solver for Coq in Elpi. In *The Coq Workshop*, July 2023. URL: https://inria.hal.science/hal-04467855.

9 Davide Fissore and Enrico Tassi. Higher-order unification for free!: Reusing the meta-language unification for the object language. In *Proceedings of PPDP*, pages 1–13. ACM, 2024. doi:10.1145/3678232.3678233.

10 Davide Fissore and Enrico Tassi. Determinacy checking for elpi: an higher-order logic programming language with cut. In *Practical Aspects of Declarative Languages: 28th International Symposium, PADL 2026, Rennes, France, January 12–13, 2026, Proceedings*, pages 77–95, Berlin, Heidelberg, 2026. Springer-Verlag. doi:10.1007/978-3-032-15981-6_5.

11 Benjamin Grégoire, Jean-Christophe Léchenet, and Enrico Tassi. Practical and sound equality tests, automatically. In *Proceedings of CPP*, page 167–181. Association for Computing Machinery, 2023. doi:10.1145/3573105.3575683.

12 Ferruccio Guidi, Claudio Sacerdoti Coen, and Enrico Tassi. Implementing type theory in higher order constraint logic programming. In *Mathematical Structures in Computer Science*, volume 29, pages 1125–1150. Cambridge University Press, 2019. doi:10.1017/S0960129518000427.

13 Robbert Krebbers, Luko van der Maas, and Enrico Tassi. Inductive Predicates via Least Fixpoints in Higher-Order Separation Logic. In Yannick Forster and Chantal Keller, editors, *16th International Conference on Interactive Theorem Proving (ITP 2025)*, volume 352 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 27:1–27:21, Dagstuhl, Germany, 2025. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. URL: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ITP.2025.27, doi:10.4230/LIPIcs.ITP.2025.27.

14 Dale Miller. A logic programming language with lambda-abstraction, function variables, and simple unification. In *Extensions of Logic Programming*, pages 253–281. Springer, 1991.

15 Dale Miller and Gopalan Nadathur. *Programming with Higher-Order Logic*. Cambridge University Press, 2012.

16 Cornelia Pusch. Verification of compiler correctness for the wam. In Gerhard Goos, Juris Hartmanis, Jan van Leeuwen, Joakim von Wright, Jim Grundy, and John Harrison, editors, *Theorem Proving in Higher Order Logics*, pages 347–361, Berlin, Heidelberg, 1996. Springer Berlin Heidelberg.

17 Enrico Tassi. Elpi: an extension language for Coq (Metaprogramming Coq in the Elpi λProlog dialect). In *The Fourth International Workshop on Coq for Programming Languages*, January 2018. URL: https://inria.hal.science/hal-01637063.

18 Enrico Tassi. Deriving proved equality tests in Coq-Elpi. In *Proceedings of ITP*, volume 141 of *LIPIcs*, pages 29:1–29:18, September 2019. URL: https://inria.hal.science/hal-01897468, doi:10.4230/LIPIcs.CVIT.2016.23.

19 Luko van der Maas. Extending the Iris Proof Mode with inductive predicates using Elpi. Master's thesis, Radboud University Nijmegen, 2024. doi:10.5281/zenodo.12568604.

20 David H.D. Warren. An Abstract Prolog Instruction Set. Technical Report Technical Note 309, SRI International, Artificial Intelligence Center, Computer Science and Technology Division, Menlo Park, CA, USA, October 1983. URL: https://www.sri.com/wp-content/uploads/2021/12/641.pdf.